Exercise sheet 6 for Numerical Optimization

Dr. Michael Lehn Tobias Born

Deadline: November 26, 2025

Exercise 1 (**Non-linear cg-method**, 7+13+4+4 points)

In the lecture *Numerical Linear Algebra*, we learned about the cg-method for solving linear equation systems

$$Ax = b \quad \text{with} \quad A \in \mathbb{R}^{n\times n},\, b \in \mathbb{R}^n \ .$$

The aim of the cg-method was to minimize the quadratic function

$$f(x) = \frac{1}{2}x^T Ax - b^T x \ .$$

In the case of a positive definite matrix A, this optimization problem admits a unique solution that also is the solution to the system of linear equations.
The cg-method can be understood as a descent method. In each iteration step, the current (negative) residual is orthogonalized to the previous residuals and used as the search direction. As the function is quadratic, the line search for the step size can be solved exactly and the optimal step size can be used. If calculated exactly, the cg-method solves the quadratic optimization problem and thus the system of linear equations in n steps. Here the algorithm for the *linear cg-method*:

Algorithm 1: Linear cg-method

1 **Input:** $A \in \mathbb{R}^{n\times n}$, $b \in \mathbb{R}^n$, initial value $x_0 \in \mathbb{R}^n$
2 **Output:** approximation x_k of the solution to the system of linear equations $x^\star = A^{-1}b$
3 $r_0 \leftarrow Ax_0 - b$
4 $d_0 \leftarrow -r_0$
5 $k \leftarrow 0$
6 **while** $\|r_k\|_2 \neq 0$ **do**
7 $\quad \alpha_k \leftarrow \frac{r_k^T r_k}{d_k^T A d_k}$
8 $\quad x_{k+1} \leftarrow x_k + \alpha_k d_k$
9 $\quad r_{k+1} \leftarrow Ax_{k+1} - b = r_k + \alpha_k A d_k$
10 $\quad \beta_{k+1} \leftarrow \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$
11 $\quad d_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} d_k$
12 $\quad k \leftarrow k + 1$

Line 11 ensures that d_{k+1} is orthogonal to all previous search directions $d_1, \ldots, d_k$ with respect to the inner product $\langle A\cdot, \cdot\rangle$. Thereby, line 11 corresponds to the Gram-Schmidt procedure, where the terms for all search directions $d_1, \ldots, d_{k-1}$ turn out to be zero.

Now we want to generalize the approach from quadratic to general smooth functions $f : \mathbb{R}^n \to \mathbb{R}$. This raises several problems. First, in the quadratic case the residual can be specified and corresponds to the gradient of f, since $f'(x) = Ax - b$. In the general case, the residual is unknown. We replace it with the gradient of f, but this no longer corresponds to the residual. This also means that the search directions are no longer orthogonal to each other. Second, the optimal

step size cannot be computed and a step size strategy (Armijo condition, Wolfe conditions, ...) has to be employed. The algorithm of the *non-linear cg-method* is:

Algorithm 2: Non-linear cg-method

1 **Input:** gradient of the function $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$, initial value $x_0 \in \mathbb{R}^n$
2 **Output:** approximation x_k of a solution to the optimization problem $\min_x f(x)$
3 compute $\nabla f_0 \leftarrow \nabla f(x_0)$
4 $d_0 \leftarrow -\nabla f_0$
5 $k \leftarrow 0$
6 **while** $\|\nabla f_k\|_2 \neq 0$ **do**
7 compute step size α_k with suitable step size strategy
8 $x_{k+1} \leftarrow x_k + \alpha_k d_k$
9 compute $\nabla f_{k+1} \leftarrow \nabla f(x_{k+1})$
10 $\beta_{k+1}^{\mathrm{FR}} \leftarrow \frac{\nabla f_{k+1}^T \nabla f_{k+1}}{\nabla f_k^T \nabla f_k}$
11 $d_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{\mathrm{FR}} d_k$
12 $k \leftarrow k + 1$

There is no reason to assume that this approach generally works well. The hope is that the function behaves like a quadratic function around a local minimum and that algorithm 2 behaves similarly to algorithm 1 there.

a) Show that algorithm 2 corresponds to algorithm 1 for quadratic functions

$$f(x) = \frac{1}{2} x^T A x - b^T x \ ,$$

assuming the step size strategy would yield the optimal step size for quadratic functions (algorithm 1, line 7).

Actually, algorithm 2 is called *non-linear cg-method according to Fletcher-Reeves*. The cg-method is generalized in various ways. The difference lies in how the factor β_{k+1} in line 11 is selected. For example, in the *non-linear cg-method according to Polak-Rebiére*, it is chosen as

$$\beta_{k+1}^{\mathrm{PR}} = \frac{\nabla f_{k+1}^T \left(\nabla f_{k+1} - \nabla f_k\right)}{\nabla f_k^T \nabla f_k}$$

and in the *non-linear cg-method according to Hestenes-Stiefel* it is

$$\beta_{k+1}^{\mathrm{HS}} = \frac{\nabla f_{k+1}^T \left(\nabla f_{k+1} - \nabla f_k\right)}{d_k^T \left(\nabla f_{k+1} - \nabla f_k\right)} \ .$$

These two methods no longer correspond to the linear cg-method in the case of a quadratic function, but they perform better in other situations.

b) Create a MATLAB program `nonlin_cg.m` that implements algorithm 2. It should be handed

- an initial value `x0`,
- a function handle `grad_f` for the function's gradient,
- a function handle `direction(grad_f_curr, grad_f_old, d)` to determine the search direction d_{k+1}, where `grad_f_curr` is the evaluation of the gradient at x_{k+1}, `grad_f_old` the evaluation of the gradient at x_k and `d` the previous search direction d_k,

- a function handle `step_size(x,d)` for the step size selection,
- a maximum number of iteration steps `max_it`
- as well as a tolerance `tol` for the termination criterion.

It is supposed to either terminate if $\|\nabla f(x_k)\|_2 <$ `tol` or if the maximum number of iteration steps is reached. Make it return a matrix of dimension ($n \times$(number iteration steps$+1$)) containing all iterates.

c) The MATLAB program `test_cg_rosenbrock.m` tests your non-linear cg-method on Rosenbrock's function using the Fletcher-Reeves variant, the Polak-Rebiére variant and the Hestenes-Stiefel variant.
First, fill the gaps for all three function handles for the determination of the search direction. Then execute the program and reflect on the results.

d) Do the same with `test_cg_himmelblau.m`.

Exercise 2 (**Global optimization: basin hopping method**, 13+3+3+3 points)

We have time for a second heuristic, global optimization approach. *Basin hopping* does exactly what the name suggests. This method attempts to jump from valley to valley on the topography of the function. In each valley, it then determines the smallest value using a local optimization approach. The goal is to find many local minima in order to obtain a value that globally is as small as possible.

The process is as follows: First, a local optimization is performed starting from the initial value. Then, starting from the local minimum, a step is taken in a random direction with the step size being random but bounded by a maximum step size. From this exploration point, again a local optimization is performed. The hope is that the exploration step reaches a new basin and therefore the local optimization can find a new local minimum. If this is successful, the iteration continues from the new local minimum. If this is not successful, a new exploration step is taken starting from the old local minimum. In order to create a certain degree of dynamics and randomness, an unsuccessful step is also accepted with a certain probability.

Algorithm 3: Basin hopping method

Input: $f : \mathbb{R}^n \to \mathbb{R}$, initial value $x_0 \in \mathbb{R}^n$, $T > 0$, a local optimization method `loc_opt(x)`, step size $h > 0$, number iteration steps *num_it*
Output: sequence of points x_{loc} trying to find as many local minima as possible
$x_{\text{loc}} \leftarrow$ `loc_opt`(x_0)
$f_{\text{loc}} \leftarrow f(x_{\text{loc}})$
$i \leftarrow 0$
while $i <$ *num_it* **do**
  $r \leftarrow (2 \cdot rand(n,1) - 1) \cdot h$
  $x_{\text{trial}} \leftarrow x_{\text{loc}} + r$
  $x_{\text{trial\_loc}} \leftarrow$ `loc_opt`(x_{trial})
  $f_{\text{trial}} \leftarrow f(x_{\text{trial\_loc}})$
  if $(f_{trial} - f_{loc} \leq 0) \vee$ *with probability* $\exp\left(-\frac{f_{trial}-f_{loc}}{T}\right)$ **then**
    $x_{\text{loc}} \leftarrow x_{\text{trial\_loc}}$
    $f_{\text{loc}} \leftarrow f_{\text{trial}}$
    $i \leftarrow i + 1$

Here, *rand(n,1)* is an n-dimensional vector containing uniformly distributed random numbers between zero and one. Thus, r is a step in a random direction with a random step size, which is however bounded by h. The counter i does not count the number of iteration steps, but the number of accepted steps.

a) Implement the basin hopping method. Therefore, fill the gaps in `basin_hopping.m`.

For the local optimization method we will use our implementation of the Hooke-Jeeves method. We could use any of our local optimizers, but for the sake of simplicity, we use a direct search method so that we do not have to deal with convergence issues of the local optimizer.

b) Adapt your implementation of the Hooke-Jeeves method such that it can be used for basin hopping.

Now we are ready to test our basin hopping. We will use the same two test cases as for the

particle swarm optimization, namely

$$f_1(x,y) = x^2 + y^2 + 10\,(2 - \cos(2\pi x) - \cos(2\pi y)) \quad \text{and}$$
$$f_2(x,y) = 1 + \frac{x^2+y^2}{4000} - \cos(x)\cos\left(\frac{y}{\sqrt{2}}\right) \ .$$

c) Execute `test_basin_hopping_f1.m` and reflect on the results.

d) Execute `test_basin_hopping_f2.m` and reflect on the results.